

Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Năm, 16 tháng 10 2025, 8:21 AM
Kết thúc lúc	Thứ Tư, 29 tháng 10 2025, 10:25 PM
Thời gian thực hiện	13 Các ngày 14 giờ
Điểm	7,00/7,00
Điểm	10,00 trên 10,00 (100%)

Câu hỏi 1

Đúng

Đạt điểm 1,00 trên 1,00

Implement static methods **Partition** and **QuickSort** in class **Sorting** to sort an array in ascending order.

```
#ifndef SORTING_H
#define SORTING_H
#include <sstream>
#include <iostream>
#include <type_traits>
using namespace std;
template <class T>
class Sorting {
private:
    static T* Partition(T* start, T* end) ;
public:
    static void QuickSort(T* start, T* end) ;
};
#endif /* SORTING_H */
```

You can read the pseudocode of the algorithm used to in method **Partition** in the below image.

ALGORITHM *HoarePartition*($A[l..r]$)
//Partitions a subarray by Hoare's algorithm, using the first element
// as a pivot
//Input: Subarray of array $A[0..n-1]$, defined by its left and right
// indices l and r ($l < r$)
//Output: Partition of $A[l..r]$, with the split position returned as
// this function's value
 $p \leftarrow A[l]$
 $i \leftarrow l; j \leftarrow r + 1$
repeat
 repeat $i \leftarrow i + 1$ **until** $A[i] \geq p$
 repeat $j \leftarrow j - 1$ **until** $A[j] \leq p$
 swap($A[i]$, $A[j]$)
until $i \geq j$
swap($A[i]$, $A[j]$) //undo last swap when $i \geq j$
swap($A[l]$, $A[j]$)
return j

For example:

Test	Result
<pre>int array[] = { 3, 5, 7, 10, 12, 14, 15, 13, 1, 2, 9, 6, 4, 8, 11, 16, 17, 18, 20, 19 }; cout << "Index of pivots: "; Sorting<int>::QuickSort(&array[0], &array[20]); cout << "\n"; cout << "Array after sorting: "; for (int i : array) cout << i << " ";</pre>	<pre>Index of pivots: 2 0 0 6 1 0 2 1 0 0 2 1 0 0 0 0 0 0 1 0 Array after sorting: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20</pre>

Answer: (penalty regime: 0 %)

Reset answer

```
1 |
2 | static T* Partition(T* start, T* end) {
3 |     T *pivot = start;
4 |     T * i = start, *j = end;
5 |
6 |     while (true){
7 |
8 |         do {
9 |             i++;
10 |         } while (*i < *pivot);
11 |
12 |         do {
13 |             j--;
14 |         } while (*j > *pivot);
15 |
```

```

16         swap (*i,*j);
17         if (i>=j) break;
18     }
19     swap (*i,*j);
20     swap(*start,*j);
21
22
23     return j;
24 }
25
26 // QuickSort cho [start, end)
27 static void QuickSort(T* start, T* end) {
28     if (start >= end) return;
29     T * pivot = Partition(start,end);
30     T * check = start;
31     int count =0;
32     while (true){
33         if(check==pivot){
34             cout<<count<<" ";
35             break;
36         }
37         count ++;
38         check ++;
39     }
40     QuickSort (start,pivot); // Sort từ start đến pivot -1
41     QuickSort(pivot+1,end); // Sort từ pivot + 1 đến end
42
43 }

```

	Test	Expected	Got	
✓	int array[] = { 3, 5, 7, 10 ,12, 14, 15, 13, 1, 2, 9, 6, 4, 8, 11, 16, 17, 18, 20, 19 }; cout << "Index of pivots: "; Sorting<int>::QuickSort(&array[0], &array[20]); cout << "\n"; cout << "Array after sorting: "; for (int i : array) cout << i << " ";	Index of pivots: 2 0 0 6 1 0 2 1 0 0 2 1 0 0 0 0 0 0 1 0 Array after sorting: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20	Index of pivots: 2 0 0 6 1 0 2 1 0 0 2 1 0 0 0 0 0 0 1 0 Array after sorting: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi 2

Đúng

Đạt điểm 1,00 trên 1,00

Implement static methods **Merge** and **MergeSort** in class Sorting to sort an array in ascending order. The Merge method has already been defined a call to method printArray so you do not have to call this method again to print your array.

```
#ifndef SORTING_H
#define SORTING_H
#include <iostream>
using namespace std;
template <class T>
class Sorting {
public:
    /* Function to print an array */
    static void printArray(T *start, T *end)
    {
        long size = end - start + 1;
        for (int i = 0; i < size - 1; i++)
            cout << start[i] << ", ";
        cout << start[size - 1];
        cout << endl;
    }

    static void merge(T* left, T* middle, T* right){
        /*TODO*/
        Sorting::printArray(left, right);
    }

    static void mergeSort(T* start, T* end) {
        /*TODO*/
    }
};
#endif /* SORTING_H */
```

For example:

Test	Result
int arr[] = {0,2,4,3,1,4}; Sorting<int>::mergeSort(&arr[0], &arr[5]);	0, 2 0, 2, 4 1, 3 1, 3, 4 0, 1, 2, 3, 4, 4
int arr[] = {1}; Sorting<int>::mergeSort(&arr[0], &arr[0]);	

Answer: (penalty regime: 0, 0, 0, 5, 10, 15, ... %)

Reset answer

```
1 static void merge(T* left, T* middle, T* right){
2     /*TODO*/
3     int n1 = middle-left +1; // Kích thước nửa trái của mảng
4     int n2 = right - middle;
5     T* L = new T[n1]; // Lưu mảng trái của phần tử
6     T *R = new T[n2]; // Lưu mảng nhánh phải của phần tử
7     // Sao chép copy dung lượng dữ liệu trong dung lượng thì triển
8     for (int i=0;i<n1;i++) L[i] = left[i];
9     for (int j=0;j<n2;j++) R[j] = middle [1+j];
10    // Xử lý cá thông số merge lại với nhau
11    int i=0,j=0;
12    T*k=left;
13    // Vị trí ghi lại
14    while (i<n1&&j<n2){
15        if(L[i] <= R[j]){
16            *k= L[i];
17            i++;
18        } else {
19            *k = R[j];
20            j++;
21        }
22        k++.
```

```

22     },
23 }
24 // Ghi 2 phần còn lại của mảng
25 while (i<n1){
26     *k =L[i];
27     i++;
28     k++;
29 }
30 // Ghi phần còn lại của mảng phải
31 while (j<n2){
32     *k=R[j];
33     j++;
34     k++;
35 }
36 Sorting::printArray(left, right);
37 delete[]L;
38 delete []R;
39 }
40 static void mergeSort(T* start, T* end){
41     /*TODO*/
42     if(start>=end) return ;
43     T *middle = start +(end-start) /2;
44     mergeSort(start,middle);
45     mergeSort(middle+1,end);
46     merge (start,middle,end);
47 }

```

	Test	Expected	Got	
✓	int arr[] = {0,2,4,3,1,4}; Sorting<int>::mergeSort(&arr[0], &arr[5]);	0, 2 0, 2, 4 1, 3 1, 3, 4 0, 1, 2, 3, 4, 4	0, 2 0, 2, 4 1, 3 1, 3, 4 0, 1, 2, 3, 4, 4	✓
✓	int arr[] = {1}; Sorting<int>::mergeSort(&arr[0], &arr[0]);			✓

Passed all tests! ✓

Đúng

Marks for this submission: 1.00/1.00.

Câu hỏi 3

Đúng

Đạt điểm 1,00 trên 1,00

The best way to sort a [singly linked list](#) given the head pointer is probably using [merge sort](#).

Both Merge sort and Insertion sort can be used for linked lists. The slow random-access performance of a linked list makes other algorithms (such as quick sort) perform poorly, and others (such as heap sort) completely impossible. Since worst case time complexity of Merge Sort is $O(n \log n)$ and Insertion sort is $O(n^2)$, merge sort is preferred.

Additionally, Merge Sort for linked list only requires a small constant amount of auxiliary storage.

To gain a deeper understanding about Merge sort on linked lists, let's implement **mergeLists** and **mergeSortList** function below

Constraints:

$0 \leq \text{list.length} \leq 10^4$

$0 \leq \text{node.val} \leq 10^6$

Use the nodes in the original list and don't modify ListNode's val attribute.

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int _val = 0, ListNode* _next = nullptr) : val(_val), next(_next) {}
};
```

// Merge two sorted lists

```
ListNode* mergeLists(ListNode* a, ListNode* b) {
```

```
// Sort an unsorted list given its head pointer
```

```
ListNode* mergeSortList(ListNode* head);
```

For example:

Test	Input	Result
<pre>int arr1[] = {1, 3, 5, 7, 9}; int arr2[] = {2, 4, 6, 8}; unordered_map<ListNode*, int> nodeAddr; ListNode* a = init(arr1, sizeof(arr1) / 4, nodeAddr); ListNode* b = init(arr2, sizeof(arr2) / 4, nodeAddr); ListNode* merged = mergeLists(a, b); try { printList(merged, nodeAddr); } catch(char const* err) { cout << err << '\n'; } freeMem(merged);</pre>		1 2 3 4 5 6 7 8 9
<pre>int size; cin >> size; int* array = new int[size]; for(int i = 0; i < size; i++) cin >> array[i]; unordered_map<ListNode*, int> nodeAddr; ListNode* head = init(array, size, nodeAddr); ListNode* sorted = mergeSortList(head); try { printList(sorted, nodeAddr); } catch(char const* err) { cout << err << '\n'; } freeMem(sorted); delete[] array;</pre>	9 9 3 8 2 1 6 7 4 5	1 2 3 4 5 6 7 8 9

Answer: (penalty regime: 0 %)

Reset answer

```
1 |
2 | ListNode* mergeLists(ListNode* a, ListNode* b) {
3 |     if (!a) return b;
4 |     if (!b) return a;
5 |
6 |     ListNode dummy(0);
7 |     ListNode* tmp = &dummy;
8 |
9 |     while (a || b) {
10 |         if (!b || (a && a->val < b->val)) {
11 |             tmp->next = a;
12 |             a = a->next;
13 |         } else {
14 |             tmp->next = b;
15 |             b = b->next;
16 |         }
17 |         tmp = tmp->next;
18 |     }
19 |     return dummy.next;
20 | }
21 |
22 | ListNode* findMid(ListNode* head) {
23 |     if (!head || !head->next) return nullptr;
24 |
25 |     ListNode* slow = head;
26 |     ListNode* fast = head->next;
27 |
28 |     while (fast && fast->next) {
29 |         slow = slow->next;
30 |         fast = fast->next->next;
31 |     }
32 |
33 |     ListNode* mid = slow->next;
34 |     slow->next = nullptr; // cắt đôi danh sách
35 |     return mid;
36 | }
37 |
38 | ListNode* mergeSortList(ListNode* head) {
39 |     if (!head || !head->next) return head;
40 |
41 |     ListNode* mid = findMid(head);
42 |     ListNode* left = mergeSortList(head);
43 |     ListNode* right = mergeSortList(mid);
44 |
45 |     return mergeLists(left, right);
46 | }
```

	Test	Input	Expected	Got	
✓	<pre>int arr1[] = {1, 3, 5, 7, 9}; int arr2[] = {2, 4, 6, 8}; unordered_map<ListNode*, int> nodeAddr; ListNode* a = init(arr1, sizeof(arr1) / 4, nodeAddr); ListNode* b = init(arr2, sizeof(arr2) / 4, nodeAddr); ListNode* merged = mergeLists(a, b); try { printList(merged, nodeAddr); } catch(char const* err) { cout << err << '\n'; } freeMem(merged);</pre>		<pre>1 2 3 4 5 6 7 8 9</pre>	<pre>1 2 3 4 5 6 7 8 9</pre>	✓

	Test	Input	Expected	Got	
✓	<pre> int size; cin >> size; int* array = new int[size]; for(int i = 0; i < size; i++) cin >> array[i]; unordered_map<ListNode*, int> nodeAddr; ListNode* head = init(array, size, nodeAddr); ListNode* sorted = mergeSortList(head); try { printList(sorted, nodeAddr); } catch(char const* err) { cout << err << '\n'; } freeMem(sorted); delete[] array; </pre>	<pre> 9 9 3 8 2 1 6 7 4 5 </pre>	<pre> 1 2 3 4 5 6 7 8 9 </pre>	<pre> 1 2 3 4 5 6 7 8 9 </pre>	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi 4

Đúng

Đạt điểm 1,00 trên 1,00

Implement static methods **merge**, **InsertionSort** and **TimSort** in class **Sorting** to sort an array in ascending order.

merge is responsible for merging two sorted subarrays. It takes three pointers: start, middle, and end, representing the left, middle, and right portions of an array.

InsertionSort is an implementation of the insertion sort algorithm. It takes two pointers, start and end, and sorts the elements in the range between them in ascending order using the insertion sort technique.

TimSort is an implementation of the TimSort algorithm, a hybrid sorting algorithm that combines insertion sort and merge sort. It takes two pointers, start and end, and an integer min_size, which determines the minimum size of subarrays to be sorted using insertion sort. The function first applies insertion sort to small subarrays, prints the intermediate result, and then performs merge operations to combine sorted subarrays until the entire array is sorted.

```
#ifndef SORTING_H
#define SORTING_H
#include <sstream>
#include <iostream>
#include <type_traits>
using namespace std;
template <class T>
class Sorting {
private:
    static void printArray(T* start, T* end)
    {
        int size = end - start;
        for (int i = 0; i < size - 1; i++)
            cout << start[i] << " ";
        cout << start[size - 1];
        cout << endl;
    }

    static void merge(T* start, T* middle, T* end) ;
public:
    static void InsertionSort(T* start, T* end) ;
    static void TimSort(T* start, T* end, int min_size) ;
};
#endif /* SORTING_H */
```

For example:

Test	Result
<pre>int array[] = { 19, 20, 18, 17 ,12, 13, 14, 15, 1, 2, 9, 6, 4, 7, 11, 16, 10, 8, 5, 3 }; int min_size = 4; Sorting<int>::TimSort(&array[0], &array[20], min_size);</pre>	<pre>Insertion Sort: 17 18 19 20 12 13 14 15 1 2 6 9 4 7 11 16 3 5 8 10 Merge 1: 12 13 14 15 17 18 19 20 1 2 6 9 4 7 11 16 3 5 8 10 Merge 2: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 3: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 4: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 5: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20</pre>

Test	Result
<pre>int array[] = { 3, 20, 18, 17, 12, 13, 14, 15, 1, 2, 9, 6, 4, 7, 11, 16, 10, 8, 5, 19 }; int min_size = 4; Sorting<int>::TimSort(&array[0], &array[20], min_size);</pre>	<pre>Insertion Sort: 3 17 18 20 12 13 14 15 1 2 6 9 4 7 11 16 5 8 10 19 Merge 1: 3 12 13 14 15 17 18 20 1 2 6 9 4 7 11 16 5 8 10 19 Merge 2: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 3: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 4: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 5: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20</pre>

Answer: (penalty regime: 0 %)

Reset answer

```

1 |
2 | static void merge(T* start, T* middle, T* end) {
3 |     int n1 = middle - start + 1;
4 |     int n2 = end - middle;
5 |
6 |     T* L = new T[n1];
7 |     T* R = new T[n2];
8 |
9 |     for (int i = 0; i < n1; i++) L[i] = *(start + i);
10 |    for (int j = 0; j < n2; j++) R[j] = *(middle + 1 + j);
11 |
12 |    int i = 0, j = 0;
13 |    T* k = start;
14 |
15 |    while (i < n1 && j < n2) {
16 |        if (L[i] <= R[j]) {
17 |            *k = L[i++];
18 |        } else {
19 |            *k = R[j++];
20 |        }
21 |        k++;
22 |    }
23 |
24 |    while (i < n1) *k++ = L[i++];
25 |    while (j < n2) *k++ = R[j++];
26 |
27 |    delete[] L;
28 |    delete[] R;
29 | }
30 | static void InsertionSort(T* start, T* end) {
31 |     for (T* i = start + 1; i < end; ++i) {
32 |         T key = *i;
33 |         T* j = i - 1;
34 |         while (j >= start && *j > key) {
35 |             *(j + 1) = *j;
36 |             j--;
37 |         }
38 |         *(j + 1) = key;
39 |     }
40 | }
41 |
42 | static void TimSort(T* start, T* end, int min_size) {
43 |     int n = end - start;
44 |
45 |     // Chia thành các đoạn nhỏ và sắp xếp bằng InsertionSort
46 |     for (int i = 0; i < n; i += min_size) {
47 |         int right = std::min(i + min_size, n);
48 |         InsertionSort(start + i, start + right);
49 |     }
50 |
51 |     std::cout << "Insertion Sort: ";
52 |     printArray(start, end);
53 |
54 |     // Ghép các đoạn nhỏ bằng merge
55 |     int merge_count = 1;
56 |     for (int size = min_size; size < n; size *= 2) {
```

```
57     for (int left = 0; left < n; left += 2 * size) {
58         int mid = std::min(left + size - 1, n - 1);
59         int right = std::min(left + 2 * size - 1, n - 1);
60         merge(start + left, start + mid, start + right);
61         std::cout << "Merge " << merge_count++ << ": ";
62         printArray(start, end);
63     }
64 }
65 }
```

	Test	Expected	Got	
✓	<pre>int array[] = { 19, 20, 18, 17 ,12, 13, 14, 15, 1, 2, 9, 6, 4, 7, 11, 16, 10, 8, 5, 3 }; int min_size = 4; Sorting<int>::TimSort(&array[0], &array[20], min_size);</pre>	<pre>Insertion Sort: 17 18 19 20 12 13 14 15 1 2 6 9 4 7 11 16 3 5 8 10 Merge 1: 12 13 14 15 17 18 19 20 1 2 6 9 4 7 11 16 3 5 8 10 Merge 2: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 3: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 4: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 5: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20</pre>	<pre>Insertion Sort: 17 18 19 20 12 13 14 15 1 2 6 9 4 7 11 16 3 5 8 10 Merge 1: 12 13 14 15 17 18 19 20 1 2 6 9 4 7 11 16 3 5 8 10 Merge 2: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 3: 12 13 14 15 17 18 19 20 1 2 4 6 7 9 11 16 3 5 8 10 Merge 4: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 5: 1 2 4 6 7 9 11 12 13 14 15 16 17 18 19 20 3 5 8 10 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20</pre>	✓
✓	<pre>int array[] = { 3, 20, 18, 17 ,12, 13, 14, 15, 1, 2, 9, 6, 4, 7, 11, 16, 10, 8, 5, 19 }; int min_size = 4; Sorting<int>::TimSort(&array[0], &array[20], min_size);</pre>	<pre>Insertion Sort: 3 17 18 20 12 13 14 15 1 2 6 9 4 7 11 16 5 8 10 19 Merge 1: 3 12 13 14 15 17 18 20 1 2 6 9 4 7 11 16 5 8 10 19 Merge 2: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 3: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 4: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 5: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20</pre>	<pre>Insertion Sort: 3 17 18 20 12 13 14 15 1 2 6 9 4 7 11 16 5 8 10 19 Merge 1: 3 12 13 14 15 17 18 20 1 2 6 9 4 7 11 16 5 8 10 19 Merge 2: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 3: 3 12 13 14 15 17 18 20 1 2 4 6 7 9 11 16 5 8 10 19 Merge 4: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 5: 1 2 3 4 6 7 9 11 12 13 14 15 16 17 18 20 5 8 10 19 Merge 6: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20</pre>	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi 5

Đúng

Đạt điểm 1,00 trên 1,00

A hotel has m rooms left, there are n people who want to stay in this hotel. You have to distribute the rooms so that as many people as possible will get a room to stay.

However, each person has a desired room size, he/she will accept the room if its size is close enough to the desired room size.

More specifically, if the maximum difference is k , and the desired room size is x , then he or she will accept a room if its size is between $x - k$ and $x + k$

Determine the maximum number of people who will get a room to stay.

input:

vector<int> rooms: rooms[i] is the size of the i th room

vector<int> people: people[i] the desired room size of the i th person

int k: maximum allowed difference. If the desired room size is x , he or she will accept a room if its size is between $x - k$ and $x + k$

output:

the maximum number of people who will get a room to stay.

Note: The iostream, vector and algorithm library are already included for you.

Constraints:

$1 \leq \text{rooms.length}, \text{people.length} \leq 2 * 10^5$

$0 \leq k \leq 10^9$

$1 \leq \text{rooms}[i], \text{people}[i] \leq 10^9$

Example 1:

Input:

rooms = {57, 45, 80, 65}

people = {30, 60, 75}

k = 5

Output:

2

Explanation:

2 is the maximum amount of people that can stay in this hotel.

There are 3 people and 4 rooms, the first person cannot stay in any room, the second and third person can stay in the first and third room, respectively

Example 2:

Input:

rooms = {59, 5, 65, 15, 42, 81, 58, 96, 50, 1}

people = {18, 59, 71, 65, 97, 83, 80, 68, 92, 67}

k = 1000

Output:

10

For example:

Test	Input	Result
<pre> int peopleCount, roomCount, k; cin >> peopleCount >> roomCount >> k; vector<int> people(peopleCount); vector<int> rooms(roomCount); for(int i = 0; i < peopleCount; i++) cin >> people[i]; for(int i = 0; i < roomCount; i++) cin >> rooms[i]; cout << maxNumberOfPeople(rooms, people, k) << '\n'; </pre>	<pre> 3 4 5 30 60 75 57 45 80 65 </pre>	2
<pre> int peopleCount, roomCount, k; cin >> peopleCount >> roomCount >> k; vector<int> people(peopleCount); vector<int> rooms(roomCount); for(int i = 0; i < peopleCount; i++) cin >> people[i]; for(int i = 0; i < roomCount; i++) cin >> rooms[i]; cout << maxNumberOfPeople(rooms, people, k) << '\n'; </pre>	<pre> 10 10 1000 18 59 71 65 97 83 80 68 92 67 59 5 65 15 42 81 58 96 50 1 </pre>	10

Answer: (penalty regime: 0 %)

Reset answer

```

1 int maxNumberOfPeople(vector<int>& rooms, vector<int>& people, int k) {
2     sort (rooms.begin(), rooms.end());
3     sort (people.begin(), people.end());
4
5     int i=0,j=0,count =0;
6     int n = rooms.size(), m = people.size();
7
8     while (i<n && j<m){
9         if (abs(rooms[i] - people [j] ) <= k){
10             count ++;
11             i++;j++;
12         } else if (rooms[i] < people [j] -k){
13             i++; // Phòng quá nhỏ chọn phòng lớn hơn
14         }
15         else {
16             j++; // Phòng quá lớn cần chọn phòng lớn hơn
17         }
18     }
19     return count;
20 }

```

	Test	Input	Expected	Got	
✓	<pre> int peopleCount, roomCount, k; cin >> peopleCount >> roomCount >> k; vector<int> people(peopleCount); vector<int> rooms(roomCount); for(int i = 0; i < peopleCount; i++) cin >> people[i]; for(int i = 0; i < roomCount; i++) cin >> rooms[i]; cout << maxNumberOfPeople(rooms, people, k) << '\n'; </pre>	<pre> 3 4 5 30 60 75 57 45 80 65 </pre>	2	2	✓
✓	<pre> int peopleCount, roomCount, k; cin >> peopleCount >> roomCount >> k; vector<int> people(peopleCount); vector<int> rooms(roomCount); for(int i = 0; i < peopleCount; i++) cin >> people[i]; for(int i = 0; i < roomCount; i++) cin >> rooms[i]; cout << maxNumberOfPeople(rooms, people, k) << '\n'; </pre>	<pre> 10 10 1000 18 59 71 65 97 83 80 68 92 67 59 5 65 15 42 81 58 96 50 1 </pre>	10	10	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1.00/1.00.

Câu hỏi 6

Đúng

Đạt điểm 1,00 trên 1,00

Given a list of distinct unsorted integers `nums`.

Your task is to implement a function with following prototype:

```
int minDiffPairs(int* arr, int n);
```

This function identify and return all pairs of elements with the smallest absolute difference among them. If there are multiple pairs that meet this criterion, the function should find and return all of them.

Note: Following libraries are included: `iostream`, `string`, `algorithm`, `sstream`

For example:

Test	Result
<pre>int arr[] = {10, 5, 7, 9, 15, 6, 11, 8, 12, 2}; cout << minDiffPairs(arr, 10);</pre>	(5, 6), (6, 7), (7, 8), (8, 9), (9, 10), (10, 11), (11, 12)
<pre>int arr[] = {10}; cout << minDiffPairs(arr, 1);</pre>	
<pre>int arr[] = {10, -1, -150, 200}; cout << minDiffPairs(arr, 4);</pre>	(-1, 10)

Answer: (penalty regime: 0 %)

Reset answer

```
1 string minDiffPairs (int *arr, int n){
2     if (n<2) return "";
3     sort(arr,arr+n);
4     int minDiff = abs(arr[1] -arr[0]);
5     for (int i=2;i<n;i++){
6         if (abs(arr[i] - arr[i-1]) < minDiff) minDiff = abs(arr[i]-arr[i-1]);
7     }
8     string str = "";
9     int count =0;
10    for (int i=1;i<n;i++){
11        if(abs(arr[i] - arr[i-1]) == minDiff){
12            if (count>0) str+=", ";
13            str+= "(" + to_string(arr[i-1]) + ", " + to_string(arr[i])+")";
14            count++;
15        }
16    }
17    return str;
18 }
```

	Test	Expected	Got	
✓	<pre>int arr[] = {10, 5, 7, 9, 15, 6, 11, 8, 12, 2}; cout << minDiffPairs(arr, 10);</pre>	(5, 6), (6, 7), (7, 8), (8, 9), (9, 10), (10, 11), (11, 12)	(5, 6), (6, 7), (7, 8), (8, 9), (9, 10), (10, 11), (11, 12)	✓
✓	<pre>int arr[] = {10}; cout << minDiffPairs(arr, 1);</pre>			✓
✓	<pre>int arr[] = {10, -1, -150, 200}; cout << minDiffPairs(arr, 4);</pre>	(-1, 10)	(-1, 10)	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi 7

Đúng

Đạt điểm 1,00 trên 1,00

Print the elements of an array in the decreasing frequency order while preserving the relative order of the elements.

Students are not allowed to use map/unordered map.

`iostream`, `algorithm` libraries are included.

For example:

Test	Result
<pre>int arr[] = {-4,1,2,2,-4,9,1,-1}; int n = sizeof(arr) / sizeof(arr[0]); sortByFrequency(arr, n); for (int i = 0; i < n; i++) cout << arr[i] << " ";</pre>	-4 -4 1 1 2 2 9 -1
<pre>int arr[] = {-5,3,8,1,-9,-9}; int n = sizeof(arr) / sizeof(arr[0]); sortByFrequency(arr, n); for (int i = 0; i < n; i++) cout << arr[i] << " ";</pre>	-9 -9 -5 3 8 1

Answer: (penalty regime: 0 %)

Reset answer

```
1 struct Array {
2     int index;    // Vị trí ban đầu của phần tử
3     int value;    // Giá trị của phần tử
4     int frequency; // Tần số xuất hiện của giá trị này
5 };
6
7 // Hàm so sánh: sắp xếp theo tần số giảm dần, nếu bằng nhau thì theo thứ tự xuất hiện
8 bool compare(const Array &a, const Array &b) {
9     if (a.frequency != b.frequency)
10         return a.frequency > b.frequency; // tần số giảm dần
11     return a.index < b.index; // nếu bằng nhau thì theo vị trí gốc
12 }
13
14 void sortByFrequency(int arr[], int n) {
15     Array array[n];
16     bool visited[n];
17
18     // Bước 1: khởi tạo
19     for (int i = 0; i < n; i++) {
20         array[i].index = i;
21         array[i].value = arr[i];
22         array[i].frequency = 0;
23         visited[i] = false;
24     }
25
26     // Bước 2: đếm tần số xuất hiện của từng phần tử
27     for (int i = 0; i < n; i++) {
28         if (visited[i])
29             continue;
30
31         int count = 1;
32         for (int j = i + 1; j < n; j++) {
33             if (arr[i] == arr[j]) {
34                 visited[j] = true;
35                 count++;
36             }
37         }
38         array[i].frequency = count;
39     }
40
41     // Bước 3: sắp xếp theo tần số giảm dần, nếu bằng nhau thì theo chỉ số gốc
```

```

42     sort(array, array + n, compare);
43
44     // Bước 4: ghi lại kết quả vào mảng đầu ra
45     int index = 0;
46     for (int i = 0; i < n; i++) {
47         for (int j = 0; j < array[i].frequency; j++)
48             arr[index++] = array[i].value;
49     }
50 }
51

```

	Test	Expected	Got	
✓	<pre> \tint arr[] = {-4,1,2,2,-4,9,1,-1}; \tint n = sizeof(arr) / sizeof(arr[0]); \tsortByFrequency(arr, n); \tfor (int i = 0; i < n; i++) \t\tcout << arr[i] << " "; </pre>	-4 -4 1 1 2 2 9 -1	-4 -4 1 1 2 2 9 -1	✓
✓	<pre> \tint arr[] = {-5,3,8,1,-9,-9}; \tint n = sizeof(arr) / sizeof(arr[0]); \tsortByFrequency(arr, n); \tfor (int i = 0; i < n; i++) \t\tcout << arr[i] << " "; </pre>	-9 -9 -5 3 8 1	-9 -9 -5 3 8 1	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1.00/1.00.